

Laboratory Report of Nanjing University of Information Science and Technology

Distributed Object Storage & Fault Tolerance Testing

1. Experimental Purpose

This experiment aims to understand distributed object storage architecture, compare traditional file storage with S3-compatible object storage, deploy a distributed MinIO cluster, observe object fragmentation, and verify fault tolerance through storage node failure simulation. Students gain practical experience with cloud-native storage technologies widely used in modern cloud platforms.

2. Experimental Environment

Operating System: Ubuntu Linux (Codespaces)

Container Platform: Docker

Orchestration Tool: Docker Compose

Object Storage Platform: MinIO

Storage Nodes: Four simulated storage disks

Client Tool: MinIO Client (mc)

3. Theoretical Background

Object storage stores data as objects rather than files and directories. Each object contains data, metadata, and a unique identifier. MinIO is an S3-compatible object storage system that supports distributed deployment and erasure coding. Erasure coding improves fault tolerance by distributing data and parity blocks across multiple disks. Even if one disk fails, the original object can still be reconstructed from remaining fragments.

Step 1: Create minio-lab directory

```
问题 输出 调试控制台 终端 端口 1

● @WentingTang → /workspaces/cloud-computing-lab-github (main) $ mkdir minio-lab
  cd minio-lab
  pwd
  /workspaces/cloud-computing-lab-github/minio-lab
○ @WentingTang → /workspaces/cloud-computing-lab-github/minio-lab (main) $
```

Step 2: Create four storage directories

```
● @WentingTang → /workspaces/cloud-computing-lab-github/minio-lab (main) $ mkdir -p data1 data2 data3 data4
  ls
  data1 data2 data3 data4
○ @WentingTang → /workspaces/cloud-computing-lab-github/minio-lab (main) $
```

Step 3: Create docker-compose.yml

```
@WentingTang → /workspaces/cloud-computing-lab-github/minio-lab (main) $ cat docker-compose.yml
- "9000:9000"
- "9001:9001"
environment:
  MINIO_ROOT_USER: admin
  MINIO_ROOT_PASSWORD: password123
command: server /data{1...4} --console-address ":9001"

mc:
  image: minio/mc
  container_name: cloud-mc
  depends_on:
    - minio
```

Step 4: Launching the Distributed MinIO Cluster

The distributed MinIO cluster was successfully started using Docker Compose. The running containers (**cloud-minio** and **cloud-mc**) confirm that the storage service and management client are operational.

```
@WentingTang → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker compose up -d
docker ps
[+] Running 3/3
✓ Network minio-lab_default Created 0.1s
✓ Container cloud-minio Started 4.4s
✓ Container cloud-mc Started 0.8s
CONTAINER ID   IMAGE          NAMES      COMMAND                  CREATED        STATUS        PORTS
715c766e8308   minio/mc      cloud-mc   "/bin/sh"               1 second ago  Up Less than a second
2224cc0c287e   quay.io/minio/minio   cloud-minio "/usr/bin/docker-ent..." 5 seconds ago  Up Less than a second 0.0.0.0:9000-9001->9000-9001/tcp, [::
]:9000-9001->9000-9001/tcp cloud-minio
○ @WentingTang → /workspaces/cloud-computing-lab-github/minio-lab (main) $
```

行 1, 列 1 空格: 4 UTF-8

Step 5: Configuring the MinIO Client Alias

The MinIO Client (**mc**) was configured with an alias named **myminio**, allowing the client to communicate with the MinIO server through the specified endpoint and authentication credentials.

```

@WentingTang →/workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123
Added 'myminio' successfully.
@WentingTang →/workspaces/cloud-computing-lab-github/minio-lab (main) $
```

Step 6: Creating a Cloud Storage Bucket

A storage bucket named **my-cloud-storage** was successfully created. Buckets are logical containers used to organize and manage objects in an object storage system.

```

Added myminio successfully.
@WentingTang →/workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc mb myminio/my-cloud-storage
Bucket created successfully `myminio/my-cloud-storage`.
@WentingTang →/workspaces/cloud-computing-lab-github/minio-lab (main) $
```

Step 7: Creating and Uploading a Large Test File

A 20 MB random binary file (**bigfile.bin**) was generated and uploaded to the cloud storage bucket. This file is used to demonstrate object distribution and fault tolerance in the MinIO cluster.

```

@WentingTang →/workspaces/cloud-computing-lab-github/minio-lab (main) $ dd if=/dev/urandom of=bigfile.bin bs=1M count=20
ls -lh bigfile.bin
docker cp bigfile.bin cloud-mc:/bigfile.bin
docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/
20971520 bytes (21 MB, 20 MiB) copied, 0.0817121 s, 257 MB/s
-rw-rw-rw- 1 codespace codespace 20M Jun 11 02:27 bigfile.bin
Successfully copied 21MB to cloud-mc:/bigfile.bin
`/bigfile.bin` -> `myminio/my-cloud-storage/bigfile.bin`
```

Total	Transferred	Duration	Speed
20.00 MiB	20.00 MiB	00m00s	27.62 MiB/s

Step 8: Verifying Object Upload

The uploaded object was successfully listed in the bucket. The presence of **bigfile.bin** confirms that the upload operation completed successfully.

```

@WentingTang →/workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc ls myminio/my-cloud-storage/
[2026-06-11 02:27:31 UTC]  20MiB STANDARD bigfile.bin
```

Step 9: Observing Distributed Storage Fragments

The backend storage directories were inspected to observe how MinIO distributes object data and metadata across multiple storage nodes. Fragmented data blocks and metadata files illustrate the distributed storage architecture.

```
@WentingTang →/workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -R data1
ls -R data2
ls -R data3
ls -R data4
data4/my-cloud-storage:
bigfile.bin

data4/my-cloud-storage/bigfile.bin:
2ec5455f-94f7-488f-ae87-01ad61e2a1ac xl.meta

data4/my-cloud-storage/bigfile.bin/2ec5455f-94f7-488f-ae87-01ad61e2a1ac:
part.1 part.2
```

Step 10: Simulating a Storage Disk Failure

One storage directory (**data3**) was intentionally deleted and recreated to simulate a storage node failure. This step tests the fault tolerance capability of the distributed storage system.

```
@WentingTang →/workspaces/cloud-computing-lab-github/minio-lab (main) $ docker compose down
sudo rm -rf data3
mkdir data3
ls
[+] Running 3/3
✓ Container cloud-mc      Removed
✓ Container cloud-minio   Removed
✓ Network minio-lab_default Removed
bigfile.bin data1 data2 data3 data4 docker-compose.yml
```

Step 11: Verifying Fault Tolerance After Disk Failure

After restarting the MinIO cluster, the uploaded object remained accessible despite the loss of one storage directory. This demonstrates the effectiveness of erasure coding and distributed redundancy mechanisms.

```
@WentingTang →/workspaces/cloud-computing-lab-github/minio-lab (main) $ docker compose up -d

docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123

docker exec cloud-mc mc ls myminio/my-cloud-storage/
[+] Running 2/2
✓ Container cloud-minio   Running
✓ Container cloud-mc      Running
Added `myminio` successfully.
[2026-06-11 02:27:31 UTC] 20MiB STANDARD bigfile.bin
```

4. Experimental Results and Analysis

The MinIO cluster was deployed successfully. The uploaded object was distributed across multiple storage directories. Metadata files and fragmented object parts were observed. After deleting one storage directory, the object remained accessible, demonstrating the effectiveness of erasure coding and distributed redundancy.

5. Reflection

Problems Encountered and Solutions:

1. Docker configuration errors were resolved by correcting YAML indentation.

2. Connection issues were solved by checking container status and aliases.
3. Upload failures were resolved by verifying bucket creation and storage availability.

Why Data Remains Available After Deleting One Storage Directory:

MinIO uses erasure coding to split data into fragments and parity blocks. When one storage node is lost, the remaining fragments contain sufficient information to reconstruct the missing data. This provides high availability, fault tolerance, and reliability in distributed cloud storage systems.

6. Conclusion

In this experiment, a distributed object storage system was successfully deployed using MinIO and Docker Compose. Four storage directories were configured to simulate a distributed storage cluster, and a cloud storage bucket was created for object management. A 20 MB test file was uploaded successfully, demonstrating the basic functionality of object storage.

By examining the backend storage structure, it was observed that MinIO automatically fragmented and distributed object data across multiple storage nodes. This process illustrates the core concept of distributed object storage, where data and metadata are stored in different locations to improve reliability and scalability.

A storage node failure was then simulated by deleting one storage directory. After restarting the cluster, the uploaded object remained accessible, proving that MinIO's erasure coding and distributed redundancy mechanisms can effectively tolerate hardware failures without causing data loss. This experiment provided a practical understanding of modern cloud storage technologies and highlighted the importance of fault tolerance, high availability, and data reliability in distributed systems.